

Manual técnico — StudIA

Versión: 1.0 · **Fecha:** 18/09/2026

Destinatario: quien tenga que mantener, corregir o extender el módulo.

Repositorio: https://github.com/MarcosDePalma/UNLZ_Llamacode_StudIA (rama `feature/studia`)

1. Principios de diseño

Cuatro reglas explican casi todas las decisiones del código. Conviene respetarlas al modificarlo:

1. **Studia es un módulo cerrado.** Todo lo propio vive en `src/core/studia/`, `qml/pages/StudiaPage.qml`, `qml/components/StudiaTemas.qml` y `tools/studia/`. No toca el agente, el chat ni los backends: sólo consume la URL del servidor que el proyecto base ya administra (`App.serverBaseUrl`, que le pasa QML).
2. **La abstención es determinística.** La decisión de no responder se toma en la recuperación, **antes** de llamar al modelo. La garantía no puede depender de que el LLM obedezca el prompt.
3. **Degradar, no romper.** Si falta el servidor de embeddings, matplotlib, mermaid-cli o Tesseract, la función asociada se apaga **avisando** y el resto sigue funcionando.
4. **El corpus es de sólo lectura.** Ninguna herramienta modifica, mueve ni borra nada del material original.

2. Mapa del código

2.1 Core C++ — `src/core/studia/` (~5.250 líneas)

Archivo	Responsabilidad	Qué NO hace
<code>StudiaIndex</code>	Búsqueda BM25 sobre FTS5, fusión híbrida RRF, reglas de evidencia	no arma prompts ni habla con el modelo
<code>StudiaPrompt</code>	Reglas, consignas de cada modo, contexto conversacional. Sin estado : todo estático	no decide cuándo abstenerse
<code>StudiaSessionStore</code>	Una conversación por materia, persistida	no sabe de recuperación
<code>StudiaController</code>	Fachada hacia QML: orquesta las anteriores y sostiene el streaming SSE	no implementa búsqueda ni formato
<code>StudiaTexto</code>	LaTeX → notación legible; separación de ecuaciones de display	no es un motor de LaTeX
<code>StudiaPlot</code>	Renderizado de gráficos, caché por MD5 del origen	no ejecuta código del modelo
<code>StudiaEmbed</code>	Vectoriza la consulta (asíncrono, no bloquea la ventana)	no vectoriza el índice (eso es offline)
<code>StudiaEmbedServer</code>	Ciclo de vida del servidor de embeddings	no elige el modelo
<code>StudiaHerramientas</code>	Qué dependencias hay y qué función habilita cada una	no instala nada por su cuenta

2.2 Herramientas Python — `tools/studia/` (~3.670 líneas)

Script	Para qué
<code>ingest.py</code>	Ingesta del corpus e índice FTS5. También <code>--archivo</code> / <code>--quitar</code> para la bibliografía propia
<code>vectorizar.py</code>	Vectorización incremental del índice
<code>ocr.py</code>	OCR de los documentos marcados <code>necesita_ocr</code>

Script	Para qué
<code>graficar.py</code>	Sidecar de matplotlib para los bloques <code>```grafico</code>
<code>estado.py</code>	Informe del índice y prueba de búsquedas
<code>listar_excluidos.py</code>	Auditoría de qué quedó afuera y por qué
<code>copiar_corpus.py</code>	Copia reducida del corpus con lo que efectivamente alimenta el índice
<code>probar_modos.py</code>	Verificación del formato de salida contra un servidor vivo
<code>instalar_dependencias.ps1</code>	Instalación idempotente de las dependencias externas

2.3 Interfaz y empaquetado

`qml/pages/StudiaPage.qml` (~1.400 líneas) concentra la interfaz; `qml/components/StudiaTemas.qml` maneja las conversaciones por materia. `installer/` contiene el script de Inno Setup, la compilación y la distribución del corpus.

3. Build y pruebas

```
build.bat Release      # aplicación
tests.bat Release      # build + suite completa (ctest)
```

Suite	Casos	Registro en ctest
<code>tests/test_studia.cpp</code> (QtTest)	226	<code>test_studia</code>
<code>tools/studia/test_ingest.py</code>	59	<code>test_studia_ingest</code>
<code>tools/studia/test_graficar.py</code>	24	<code>test_studia_graficar</code>
<code>tools/studia/test_vectorizar.py</code>	13	<code>test_studia_vectorizar</code>

Regla del proyecto base: build más suite en verde es la condición para commitear.

Si no hay Python en el sistema, CMake omite las suites en Python y el resto sigue corriendo; las que además necesitan numpy o matplotlib se saltan solas.

3.1 Probar lo que los tests no pueden probar

Un test de C++ verifica que la instrucción **esté en el prompt**; no dice nada sobre si el modelo la cumple. Para eso:

```
tests.bat Release      # vuelca los prompts a %TEMP%
python tools/studia/probar_modos.py "Redes" "modelo OSI capas" "el modelo OSI"
```

Al tocar los modos con formato estricto (Autoevaluación, Flashcards, Plan) correrlo sobre tres o cuatro materias antes de dar el cambio por bueno. Dos resultados ya verificados que conviene no volver a intentar:

- El formato puesto **sólo** en el prompt de sistema **se ignora**: tiene que ir al final del mensaje de usuario.
- Pedir una estructura global ("primero las 10 preguntas, después las 10 respuestas") **no se sostiene**; los pares `P:` / `R:` sí.

4. Configuración y rutas

Clave (<code>QSettings</code>)	Contenido
<code>studia/rutaIndice</code>	Ruta de <code>studia.db</code>
<code>studia/carpetaDocumentos</code>	Raíz del corpus, para abrir los originales desde las citas
<code>studia/materia</code>	Última materia seleccionada
<code>studia/urlEmbeddings</code>	URL del servidor de embeddings (vacío = sólo búsqueda léxica)

Dato	Ubicación
Conversaciones	<code>AppLocalData/LlamaCode/studia/</code>
Bibliografía propia	<code>AppLocalData/LlamaCode/studia/mi_biblioteca.db</code>
Caché de gráficos	<code>AppLocalData/LlamaCode/studia/graficos/</code>

En los tests, `QStandardPaths::setTestModeEnabled(true)` redirige esas ubicaciones. La suite de StudIA **aísla además sus `QSettings`** : sin eso escribía en el registro real del usuario.

5. Tareas de mantenimiento frecuentes

5.1 Recalibrar el umbral de abstención

Obligatorio con cada corpus nuevo. Los valores de BM25 dependen del tamaño del índice: un umbral calibrado para 139.000 fragmentos no sirve para otro corpus.

1. Armar un conjunto de preguntas en lenguaje natural, mitad cubiertas por el material y mitad ajenas (el original tenía 30: 16 y 14).
2. Probar con `estado.py --buscar` y observar los scores de ambos grupos.
3. Ajustar `umbralAbstencion` (default `-7.0`) al valor que responde todas las cubiertas y rechaza todas las ajenas.
4. Correr la suite: hay tests que fijan el comportamiento esperado del control.

5.2 Agregar un modo de tutor

1. Sumar la entrada en `modos()` de `StudiaPrompt.cpp` : id, etiqueta, color y si es **exigente** o **flexible** (§6.4 del informe).
2. Escribir la consigna del modo y, si el formato importa, el recordatorio que va al final del mensaje de usuario.
3. Agregar el test correspondiente en `tests/test_studia.cpp`.
4. Verificar el formato real con `probar_modos.py` sobre varias materias.

Si se renombra un id existente, agregarlo como **alias** en `idCanónico()` : hay conversaciones guardadas con el id anterior que si no se quedan sin etiqueta ni color.

5.3 Sumar un formato de documento a la ingesta

1. Agregar la extensión a `EXT_SOPORTADAS` en `ingest.py`.
2. Escribir la función `extraer_<formato>` y registrarla en el mapa de extractores.
3. Agregar los casos en `test_ingest.py`, incluyendo el archivo vacío y el corrupto.
4. Verificar que el documento sin texto extraíble quede marcado `necesita_ocr` y no descartado.

5.4 Cambiar el modelo de embeddings

El modelo empaquetado es **bge-m3**, que requiere `--pooling cls` en el servidor: con otro pooling los vectores salen mal sin dar error. Al cambiarlo hay que **re-vectorizar el índice completo**; los vectores de otra dimensión se ignoran en vez de producir resultados sin sentido, y `vectores_info` registra dimensión y modelo para poder detectarlo.

6. Puntos delicados

Tema	Qué hay que saber
Fusión híbrida	Se fusionan rangos (RRF), no puntajes: BM25 y el coseno no son escalas comparables. No "mejorar" esto promediando scores.
Dos índices	El de cátedra y el propio se consultan por separado, con cupo para el propio. El propio corre sin exigir evidencia porque en un índice de pocos fragmentos el score BM25 se anula.
Normalización del score	Se divide por la cantidad de términos discriminantes , no por el total: dividir por el total castiga a las preguntas en lenguaje natural.
Orden de materias	El cuatrimestre es texto (<code>"1C".."10C"</code>): hay que castear a entero o <code>10C</code> queda antes que <code>1C</code> .
Ecuaciones	Un renglón entre corchetes sin LaTeX (una cita, una lista) no debe tomarse como ecuación de display.
Graficador	Evalúa la expresión en un espacio de nombres cerrado, sin <code>__builtins__</code> . Nunca ejecutar código Python emitido por el modelo.
Perfiles	La corrección de perfiles duplicados vive en <code>ProfileManager</code> (proyecto base) y migra los perfiles existentes: no revertirla al sincronizar con upstream.

7. Empaquetado

`installer\compilar.bat` arma con Inno Setup lo que se entrega; sale a `dist\` (ignorado por git).

- **Aplicación** (`StudIA.iss`): un `.exe` de 1,21 GB con la app, `studia.db` y el modelo de embeddings. Instala sin privilegios de administrador y registra la ruta en `HKA\Software\StudIA\InstallDir`.
- **Documentos** (`copiar_documentos.ps1`): 7,7 GB aparte, por `robocopy` a un destino de ruta corta.

Dos cosas que hay que mantener y no son obvias:

1. `compilar.ps1` **copia el runtime de MSVC** junto al ejecutable. Sin eso la aplicación no abre en una PC sin Visual Studio. `llama-server.exe` vive en otra carpeta y no alcanza esas copias: de ese se ocupa `instalar_dependencias.ps1`.

2. `compilar.ps1` copia los scripts del repositorio a `build\Release\StudIA` antes de empaquetar, para no publicar la versión vieja de alguno que se haya tocado.

8. Relación con el proyecto base

`main` sigue a `cristianlukas/UNLZ_Llamacode`; el desarrollo vive en `feature/studia`. Para incorporar cambios de upstream:

```
git fetch upstream
git checkout main && git merge upstream/main
git checkout feature/studia && git merge main
build.bat Release && tests.bat Release
```

Estado medido al 27/08/2026: `main` está **583 commits** por delante del punto en que se hizo el fork, y un merge de `feature/studia` deja **7 archivos en conflicto**:

Archivo	Motivo
<code>CMakeLists.txt</code>	fuentes y tests nuevos contra la reorganización de upstream
<code>qml/components/NavBar.qml</code>	la entrada de StudIA contra las de upstream
<code>qml/Main.qml</code> · <code>src/AppController.cpp</code>	registro del módulo
<code>CLAUDE.md</code> · <code>.gitignore</code>	secciones agregadas
<code>LlamaCode.lnk</code>	eliminado en upstream, modificado en la rama

Ninguno es un conflicto de lógica: son puntos de registro del módulo. Aun así, después de resolverlos hay que **recompilar y correr la suite completa** antes de dar el merge por bueno.

Para comprobar el estado antes de intentarlo:

```
git merge-tree --write-tree main feature/studia | grep CONFLICT
```

9. Deudas conocidas

Deuda	Detalle
Validación en VM limpia	Procedimiento escrito, ejecución pendiente (<code>installer/PROBAR_EN_PC_LIMPIA.md</code>)
154 documentos sin OCR	Identificados y registrados; falta procesarlos y re-vectorizar
Backends con SSE real sin cobertura	Limitación heredada del proyecto base: los tests cubren sesiones y persistencia, no la red
Umbral atado al corpus	No hay recalibración automática al cambiar de corpus
Sólo Windows	La versión empaquetada; el core no tiene dependencias específicas de plataforma más allá del build